

**University of North Carolina at Charlotte
Department of Electrical and Computer Engineering**

Homework Report 1

ECGR 5106 – Introduction to Deep Learning

Name: Viprav Lipare

Student #: 810288922

Date: June 12th, 2026

GitHub: <https://github.com/vipravlipare/ECGR-5106-Intro-To-Deep-Learning>

Homework 1 Report

Problem 1: Modified AlexNet on CIFAR-10

Objective

AlexNet was originally designed for 227x227 ImageNet images with around 61 million parameters. It is too large for CIFAR-10's 32x32 images, where spatial dimensions collapse too quickly through the pooling layers. Your task is to simplify AlexNet for CIFAR-10, analyze the result, and explore dropout as a regularization strategy.

Part A: Simplified Architecture

Model Setup

The convolution part of my model is:

- Conv2d(3, 32, kernel_size=3, padding=1) -> BatchNorm -> ReLU -> MaxPool
- Conv2d(32, 64, kernel_size=3, padding=1) -> BatchNorm -> ReLU -> MaxPool
- Conv2d(64, 128, kernel_size=3, padding=1) -> BatchNorm -> ReLU
- Conv2d(128, 128, kernel_size=3, padding=1) -> BatchNorm -> ReLU
- Conv2d(128, 64, kernel_size=3, padding=1) -> BatchNorm -> ReLU -> MaxPool

The fully connected part is:

- Flatten
- Linear(64 * 4 * 4, 256) -> ReLU
- Linear(256, 128) -> ReLU
- Linear(128, 10)

Parameter Count:

- Trainable parameters: 612,042
- Original AlexNet parameters: about 61,000,000
- Just above 1% of the total parameters were used, this is because training on CIFAR-10 is much simpler than training on ImageNet

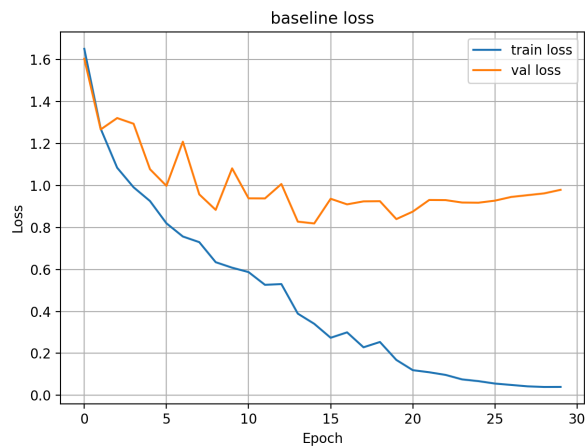
Training Setup and Results

I trained the baseline for 30 epochs using seed 42, AdamW, learning rate 0.001, batch size 256, and ReduceLROnPlateau. I used RandomHorizontalFlip for training and CIFAR-10 normalization for train, validation, and test data. Since the run was on my laptop CPU, I used fast_cpu=True and used 10,000 training images and 2,000 validation images while keeping the full 10,000-image CIFAR-10 test set.

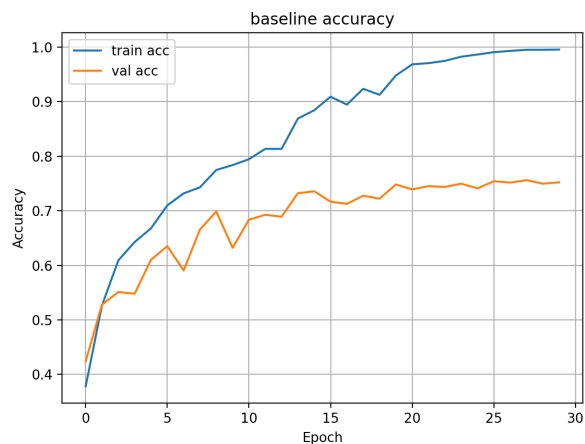
The baseline reached 75.60% best validation accuracy and 75.33% test accuracy. The final training accuracy was 99.52%, so the model learned the training subset very strongly and showed major overfitting. The first-layer filters mostly show small color, and contrast. The confusion matrix was strongest for ship, frog, automobile, truck, airplane, and horse, while most errors were between similar animal classes such as cat and dog.

Required Part A results: The baseline ended with final train loss 0.0401, final validation loss 0.9796, best validation accuracy 75.60%, final validation accuracy 75.20%, and final test accuracy 75.33%. The figures below include the training/validation loss curve, accuracy curve, first-layer filter visualization, and confusion matrix.

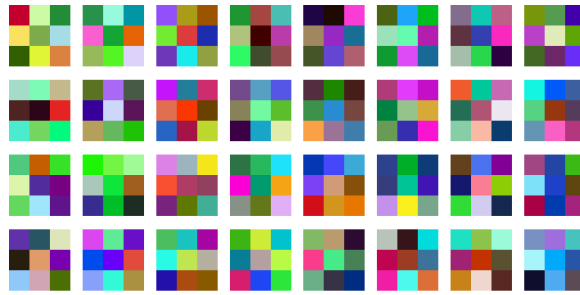
Plots



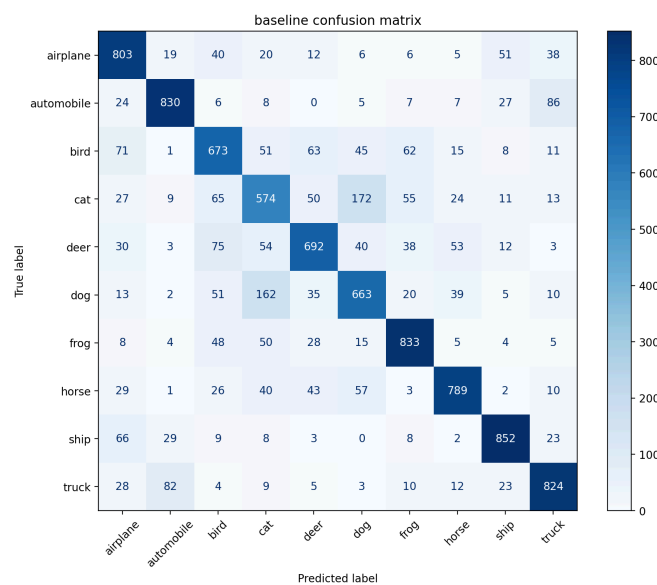
AlexNet baseline loss curve



AlexNet baseline accuracy curve



First-layer AlexNet filters



AlexNet confusion matrix

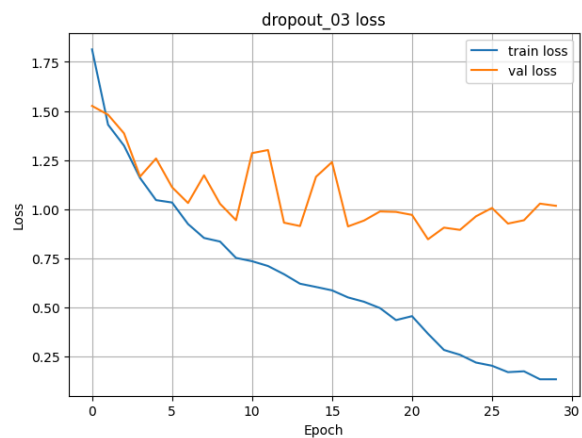
Part B: Dropout

I retrained the same modified AlexNet with dropout $p=0.3$ and $p=0.5$ in the fully connected layers. All other settings were kept the same as Part A. The $p=0.3$ model gave the best test accuracy at 75.66%, while $p=0.5$ gave 75.52% and the no-dropout baseline gave 75.33%.

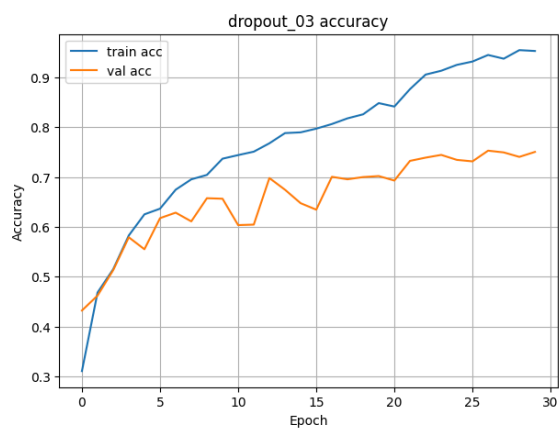
Dropout lowered the training accuracy, which means it increased generalization. The validation accuracy stayed close for all three runs, so dropout helped only slightly. I would choose $p=0.3$ because it had the best test accuracy and a smaller training/validation gap than the baseline.

Required Part B results: The best dropout model was $p=0.3$. It used the same hyperparameters as Part A and reached final train loss 0.1350, final validation loss 1.0177, best validation accuracy 75.30%, final validation accuracy 75.05%, and final test accuracy 75.66%. The dropout comparison plot and best-dropout confusion matrix are included below.

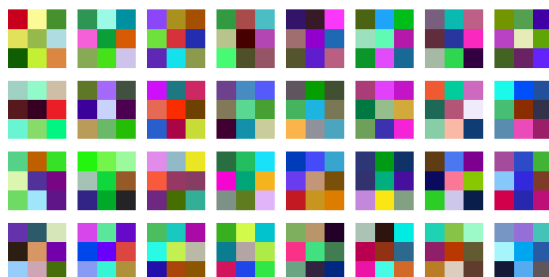
Dropout (0.3) Plots



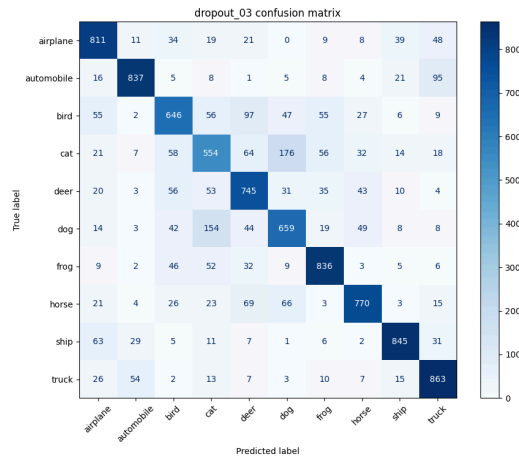
AlexNet Dropout=0.3 loss curve



AlexNet Dropout=0.3 accuracy curve

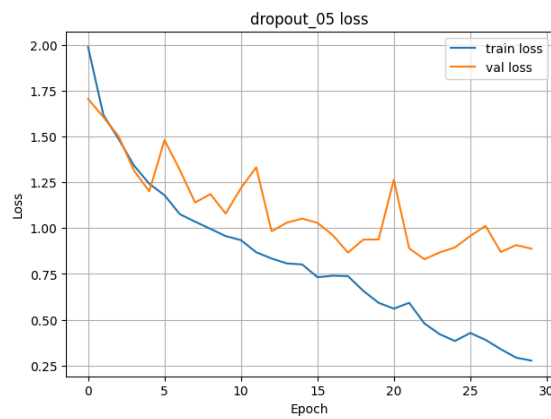


Dropout = 0.3 AlexNet filters

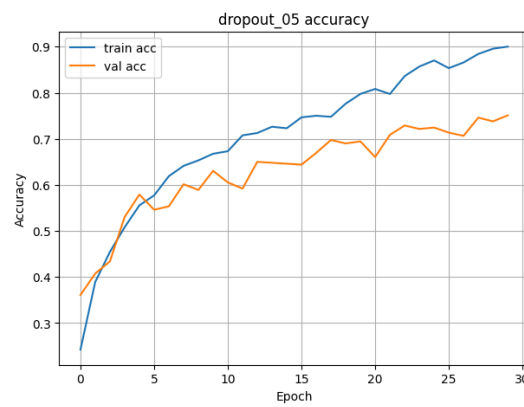


Dropout = 0.3 Confusion Matrix

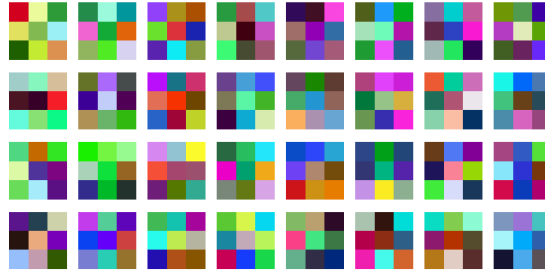
Dropout (0.5) Plots



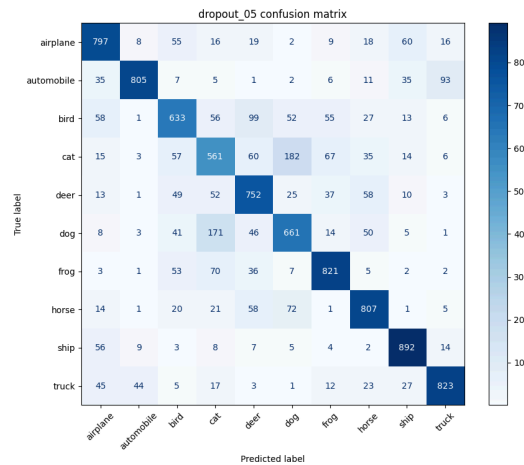
AlexNet Dropout=0.5 loss curve



AlexNet Dropout=0.5 accuracy curve



Dropout = 0.5 AlexNet filters



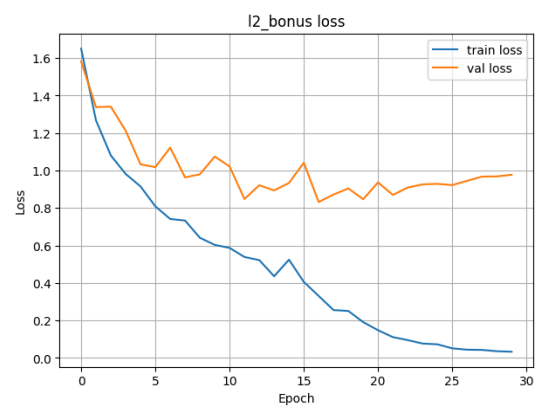
Dropout = 0.5 Confusion Matrix

Bonus: L2 Weight Decay

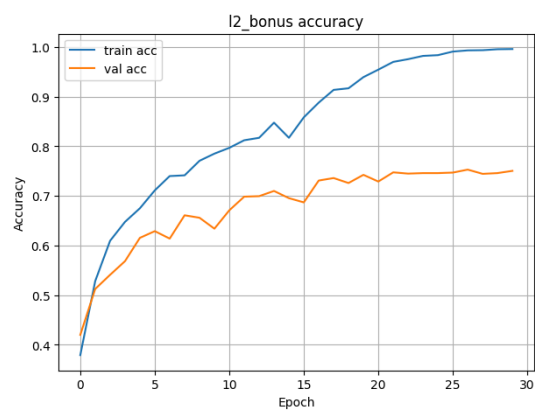
For the bonus, I also tested L2 weight decay with `weight_decay=0.0001`. It reached 75.23% test accuracy and 75.30% best validation accuracy. This was close to the baseline, but it did not beat dropout $p=0.3$. In this setup, L2 was stable, but dropout was the better regularization choice.

Required Bonus results: The L2 run used `weight_decay=0.0001` with the same baseline settings. It ended with final train loss 0.0333, final validation loss 0.9773, best validation accuracy 75.30%, final validation accuracy 75.05%, and final test accuracy 75.23% on CPU.

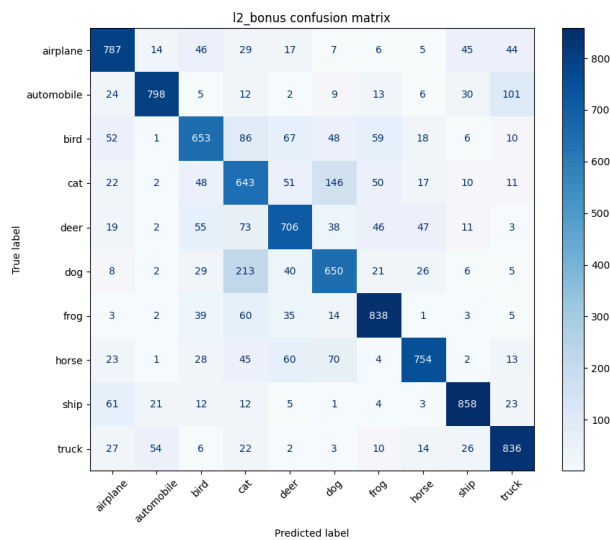
Plots



AlexNet L2 Bonus loss curve



AlexNet L2 Bonus Accuracy curve



AlexNet L2 Bonus Confusion Matrix

Problem 2: Modified VGGNet on CIFAR-10

Part A: Adapted VGGNet

Model Setup

The convolution part of my VGG model is:

- Conv2d(3, 32) -> Conv2d(32, 32) -> MaxPool
- Conv2d(32, 64) -> Conv2d(64, 64) -> MaxPool
- Conv2d(64, 128) -> Conv2d(128, 128) -> MaxPool
- Conv2d(128, 128) -> MaxPool

The fully connected part is:

- Flatten
- Linear(128 * 2 * 2, 256) -> ReLU
- Linear(256, 128) -> ReLU
- Linear(128, 10)

Parameter Count:

- Trainable parameters: 601,258
- Modified AlexNet parameters: 612,042

This VGG version was picked because its parameter count is very close to my AlexNet model, so the comparison is more fair. I used a reduced VGG-style model closest to VGG-11/VGG-13 because its parameter count, 601,258, was closest to my AlexNet model at 612,042.

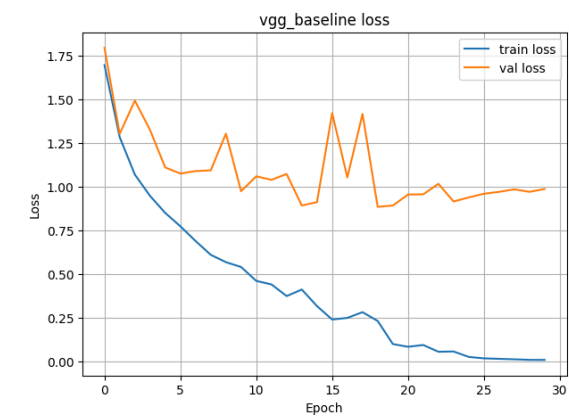
Training Setup and Results

I trained the VGG baseline for 30 epochs using seed 42, AdamW, learning rate 0.001, batch size 256, and ReduceLROnPlateau. I used the same CIFAR-10 split, normalization, and RandomHorizontalFlip augmentation as Problem 1.

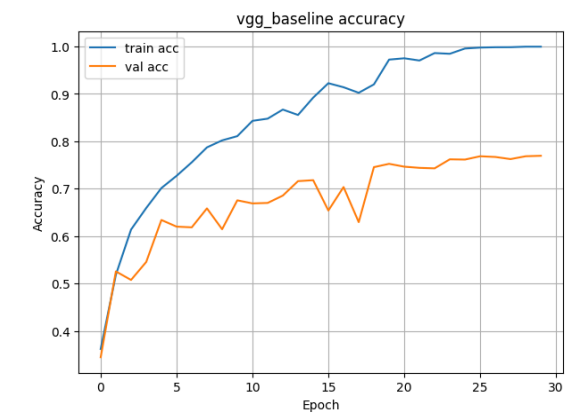
The baseline VGG reached 76.95% best validation accuracy and 77.72% test accuracy. This was better than the AlexNet baseline, but it trained much slower because VGG uses more repeated convolution layers.

Required Part A results: The VGG baseline ended with final train loss 0.0070, final validation loss 0.9871, best validation accuracy 76.95%, final validation accuracy 76.95%, and final test accuracy 77.72%.

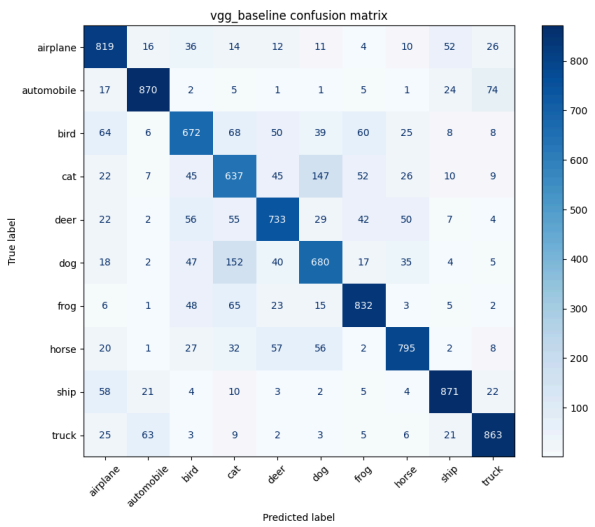
Plots



VGG baseline loss curve



VGG baseline accuracy curve



VGG confusion matrix

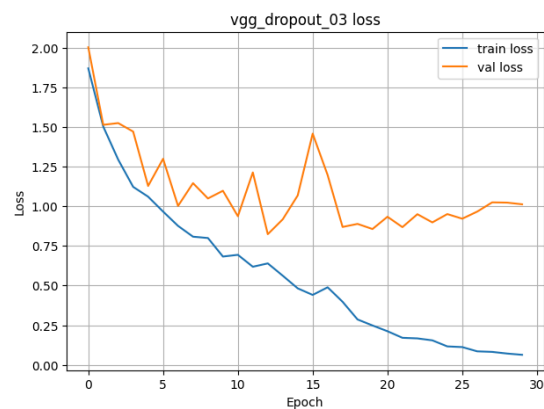
Part B: Dropout and Cross-Model Comparison

I retrained the same VGG model with dropout $p=0.3$ and $p=0.5$ in the fully connected layers. All other settings were kept the same as Part A.

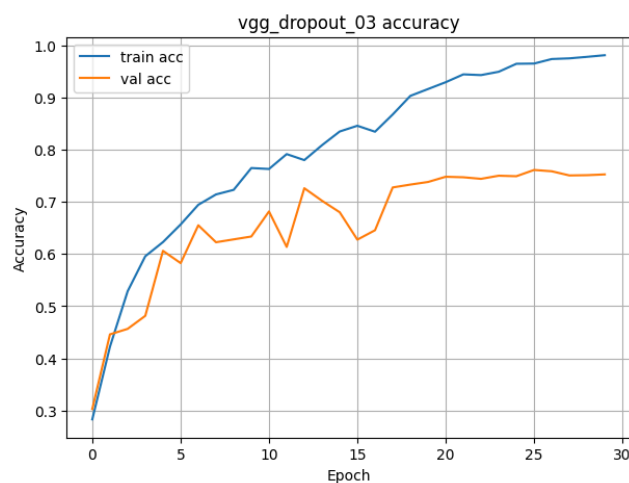
The $p=0.5$ model gave the best test accuracy at 78.05%, while $p=0.3$ gave 77.39% and the no-dropout VGG baseline gave 77.72%. Dropout helped the VGG model a little more than it helped AlexNet, but the improvement was still small.

Required Part B results: The best VGG dropout model was $p=0.5$. It reached final train loss 0.1942, final validation loss 0.9310, best validation accuracy 74.90%, final validation accuracy 74.90%, and final test accuracy 78.05%.

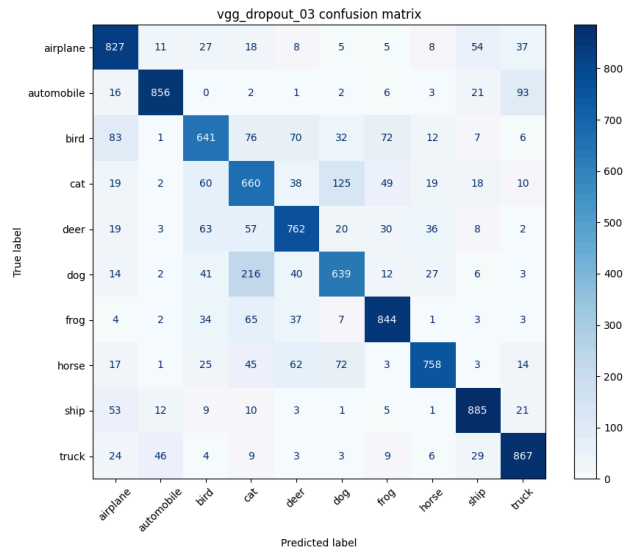
Plots (Dropout - 0.3)



VGG 0.3 Dropout loss curve

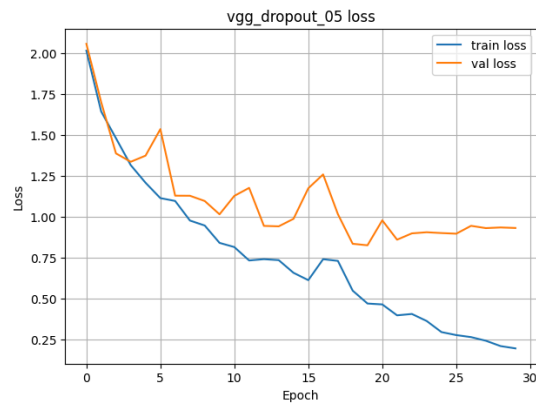


VGG 0.3 Dropout accuracy curve

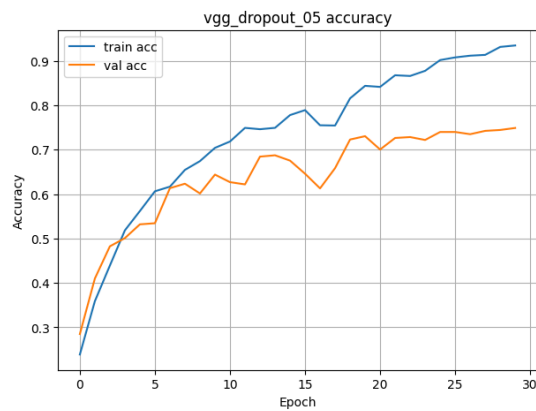


VGG 0.3 Dropout confusion matrix

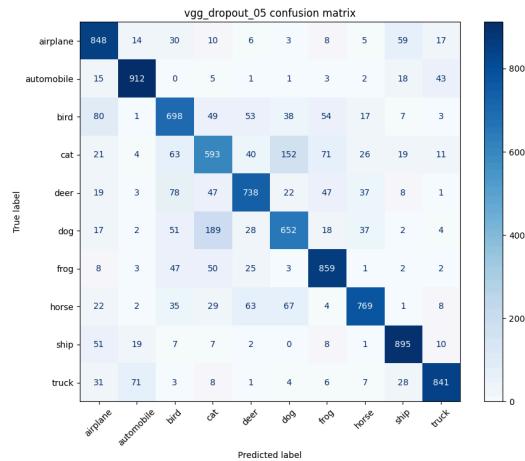
Plots (Dropout - 0.5)



VGG 0.5 Dropout loss curve



VGG 0.5 Dropout accuracy curve



VGG 0.5 Dropout confusion matrix

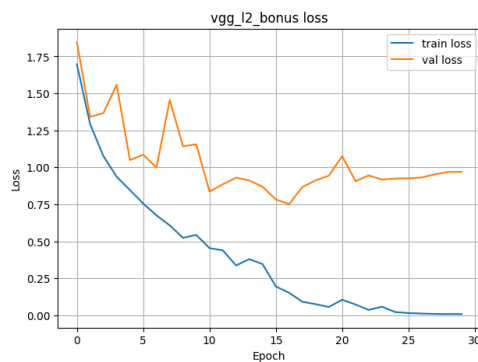
Bonus: Batch Normalization

For the bonus, BatchNorm was included after each convolution layer in the VGG model. This helped keep the deeper VGG model stable while training.

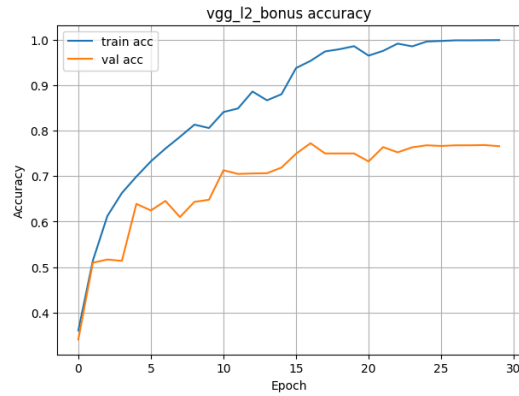
I also saved a bonus regularization run with `weight_decay=0.0001`. It reached 77.26% test accuracy and 77.25% best validation accuracy. This was close to the VGG baseline, but it did not beat the dropout $p=0.5$ result.

Required Bonus results: The bonus run ended with final train loss 0.0092, final validation loss 0.9700, best validation accuracy 77.25%, final validation accuracy 76.60%, and final test accuracy 77.26%.

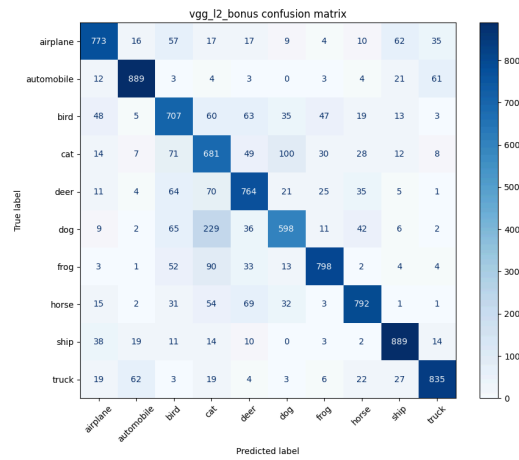
Plots



VGG BatchNorm loss curve



VGG BatchNorm accuracy curve



VGG BatchNorm confusion matrix

Model Results

Model	Parameters	Best Val Acc	Test Acc	Best Epoch	Time	Sec/Epoch
vgg_baseline	601258	76.95%	77.72%	30	36.10 min	72.20 sec
vgg_dropout_03	601258	76.10%	77.39%	26	42.55 min	85.10 sec
vgg_dropout_05	601258	74.90%	78.05%	30	41.25 min	82.49 sec
vgg_l2_bonus	601258	77.25%	77.26%	17	43.33 min	86.66 sec

AlexNet vs VGGNet

Architecture	Best Variant	Parameters	Test Acc	Training Time	Sec/Epoch
AlexNet	dropout_03	612042	75.66%	9.51 min	19.03 sec
VGGNet	vgg_dropout_05	601258	78.05%	41.25 min	82.49 sec

Problem 3: ResNet-11 vs ResNet-18 on CIFAR-10

Part A: ResNet-18 Implementation

Model Setup

I implemented ResNet-18 from scratch using BasicBlocks. Each block has two 3x3 convolution layers with BatchNorm and ReLU, and then the input is added back using a skip connection.

The ResNet-18 setup is:

- 3x3 starting convolution, no 7x7 convolution
- No first max-pooling layer for CIFAR-10
- Four residual stages with [2, 2, 2, 2] blocks
- Global average pooling
- Final fully connected layer with 10 outputs

Parameter Count:

- ResNet-11 parameters: 604,250
- ResNet-18 parameters: 701,466

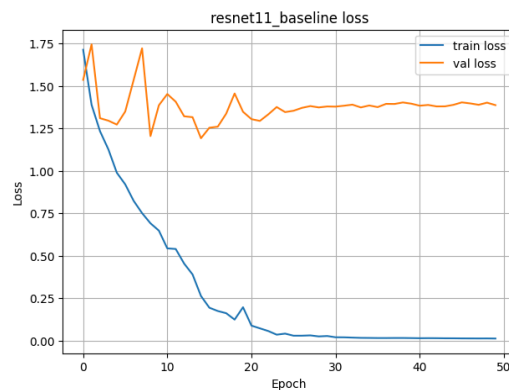
Training Setup and Results

I trained the ResNet models for 50 epochs using seed 42, AdamW, learning rate 0.001, batch size 256, ReduceLROnPlateau, and CPU hardware. I used the same CIFAR-10 split, normalization, and augmentation as the earlier problems.

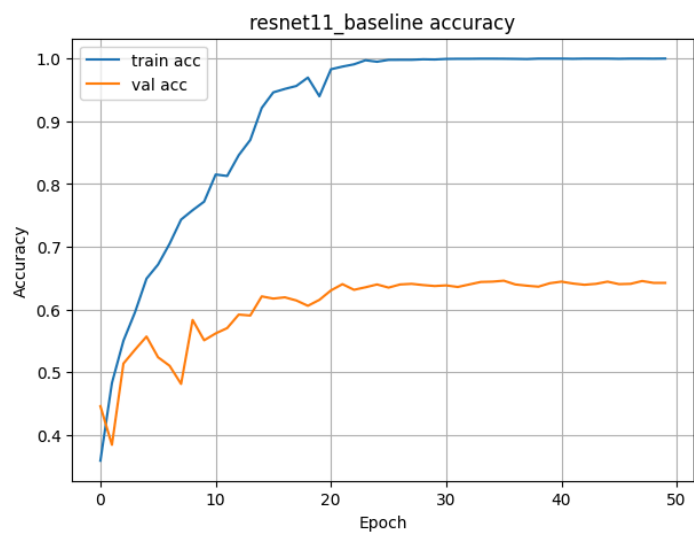
The ResNet-18 baseline reached 69.05% best validation accuracy and 68.49% test accuracy. ResNet-11 reached 64.59% test accuracy. ResNet-18 did better, but it also took much longer to train.

Required Part A results: ResNet-18 baseline ended with final train loss 0.0110, final validation loss 1.2362, best validation accuracy 69.05%, final validation accuracy 68.20%, and final test accuracy 68.49%.

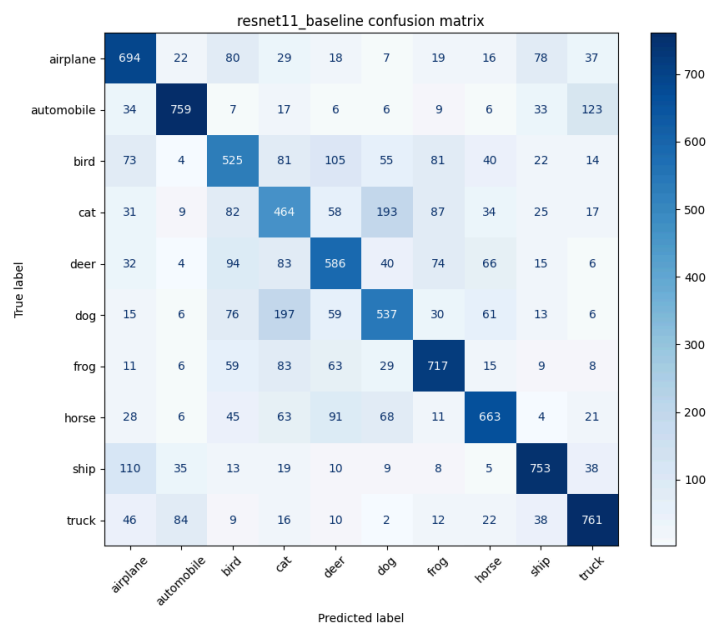
Plots (ResNet 11)



ResNet 11 baseline loss curve

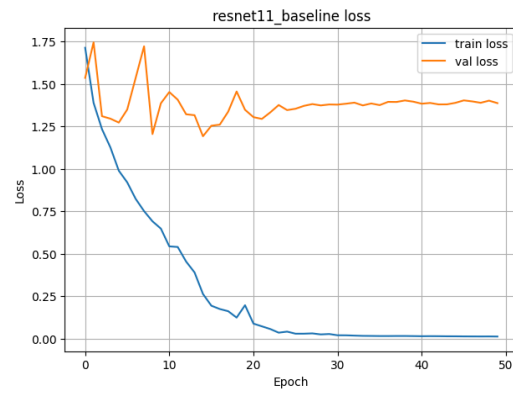


ResNet 11 baseline accuracy curve

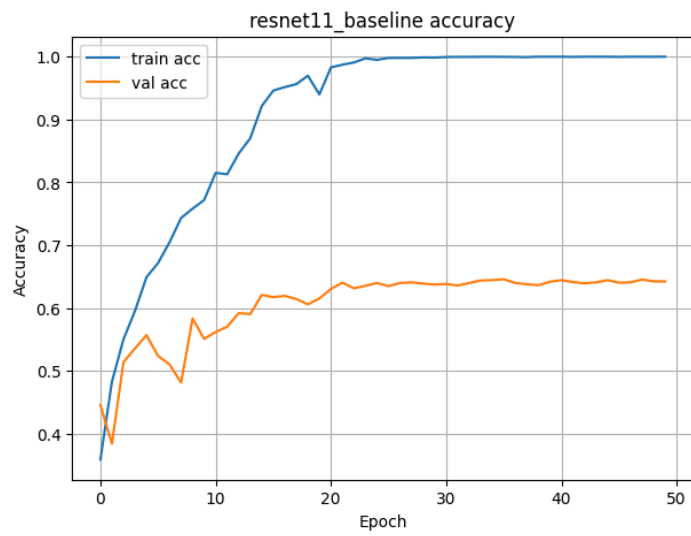


ResNet 11 confusion matrix

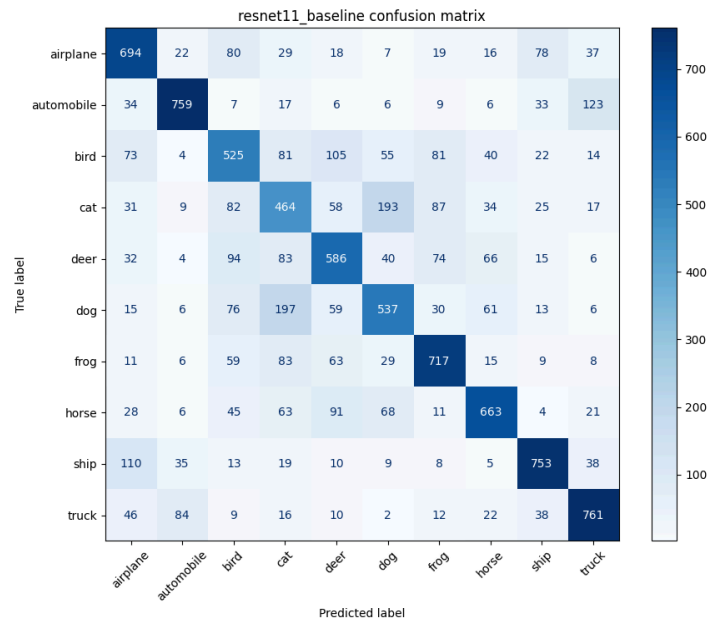
Plots (ResNet 11)



ResNet 11 baseline loss curve

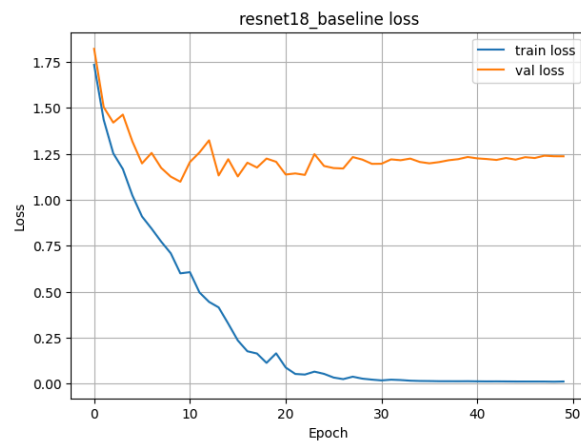


ResNet 11 baseline accuracy curve

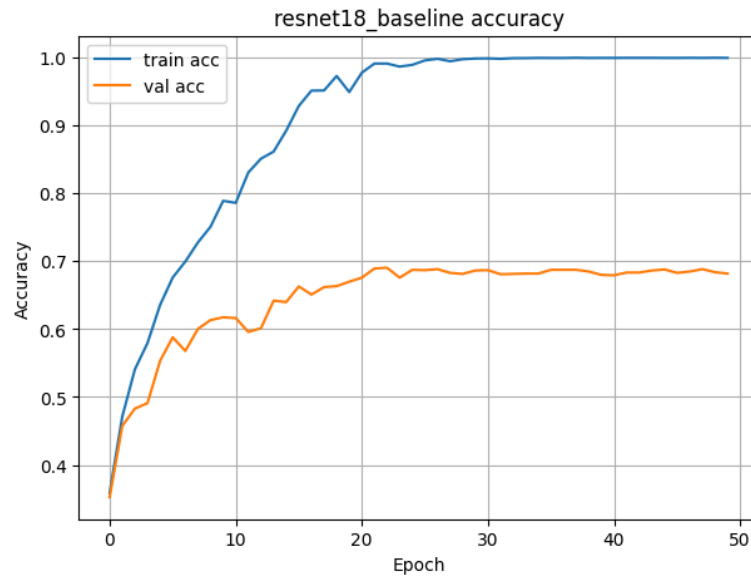


ResNet 11 confusion matrix

Plots (ResNet 18)



ResNet 18 baseline loss curve



ResNet 18 baseline accuracy curve



ResNet 18 confusion matrix

Part B: Dropout and Final Comparison

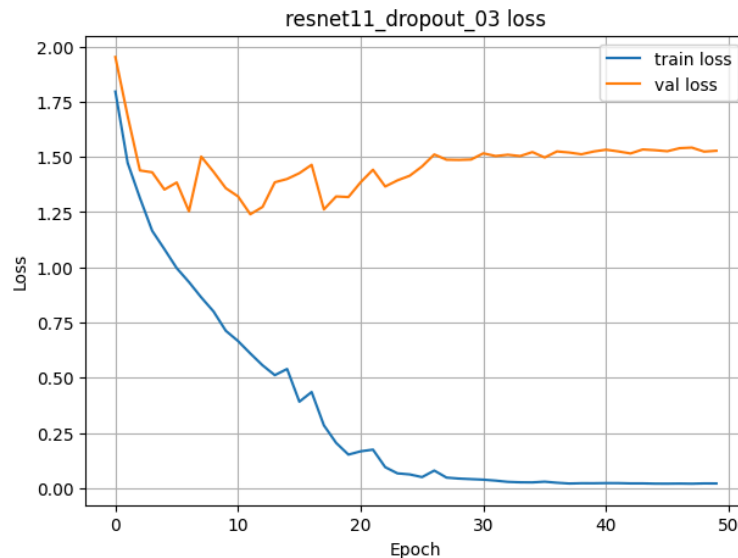
I tested dropout $p=0.3$ and $p=0.5$ after global average pooling for both ResNet-11 and ResNet-18. All other settings were kept the same.

For ResNet-11, the baseline test accuracy was 64.59%, $p=0.3$ gave 63.40%, and $p=0.5$ gave 64.27%. For ResNet-18, the baseline test accuracy was 68.49%, $p=0.3$ gave 66.53%, and $p=0.5$ gave 68.17%.

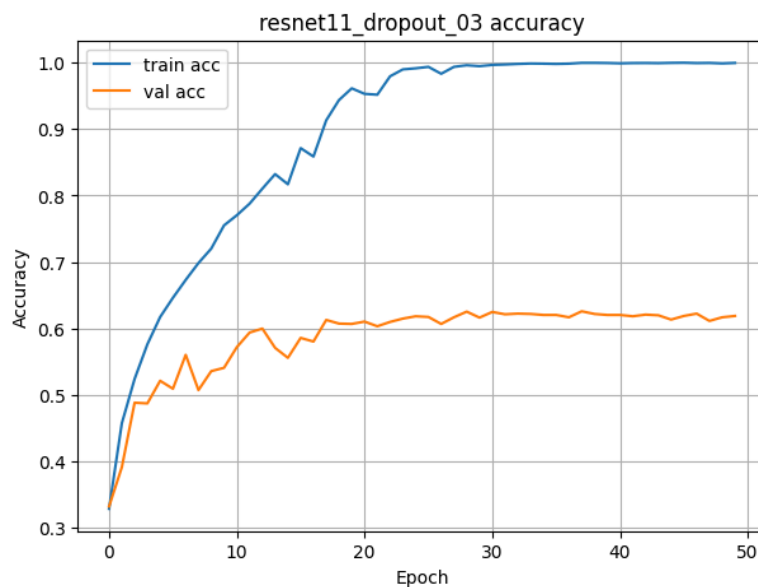
Dropout did not really help the ResNet models. This is probably because ResNet already uses BatchNorm inside the residual blocks, so adding dropout after pooling did not give much extra benefit.

Required Part B results: The best ResNet dropout result was ResNet-18 with $p=0.5$. It ended with final train loss 0.0214, final validation loss 1.3149, best validation accuracy 68.20%, final validation accuracy 67.60%, and final test accuracy 68.17%.

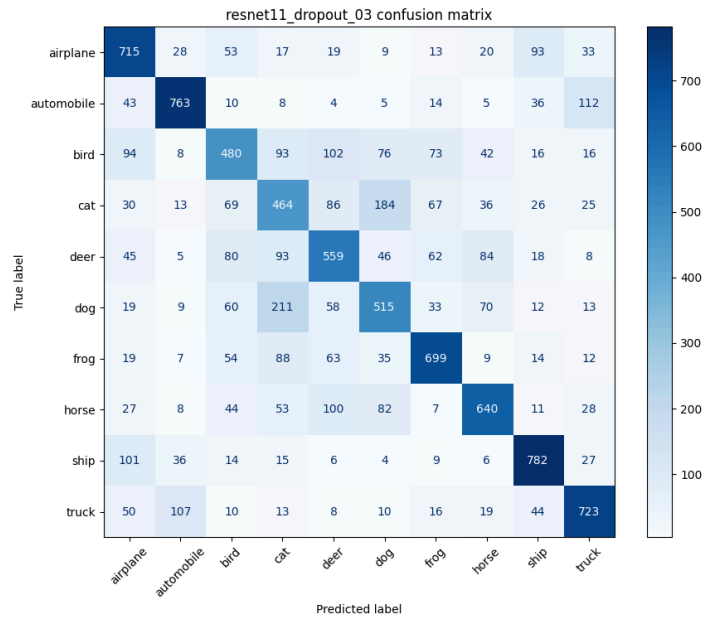
Plots (ResNet 11 - Dropout 0.3)



ResNet 11 - Dropout 0.3 baseline loss curve

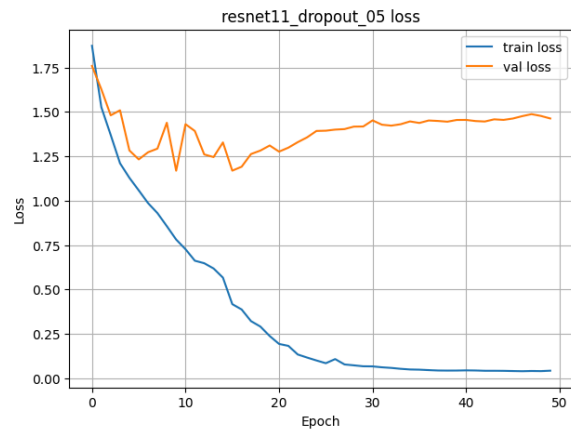


ResNet 11 - Dropout 0.3 baseline accuracy curve

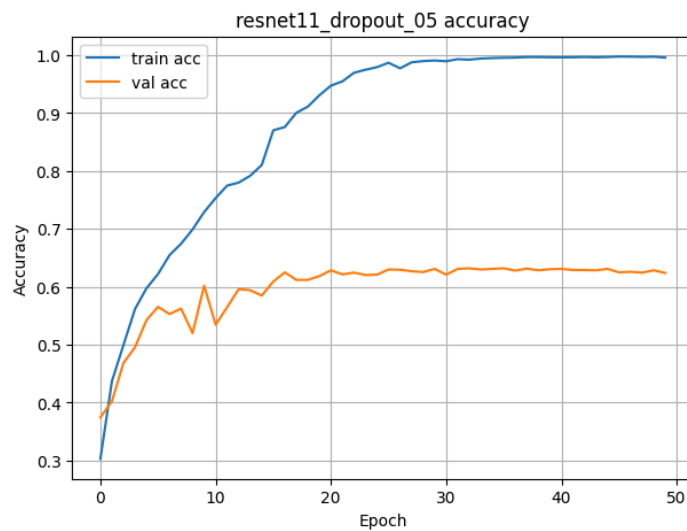


ResNet 11 - Dropout 0.3 confusion matrix

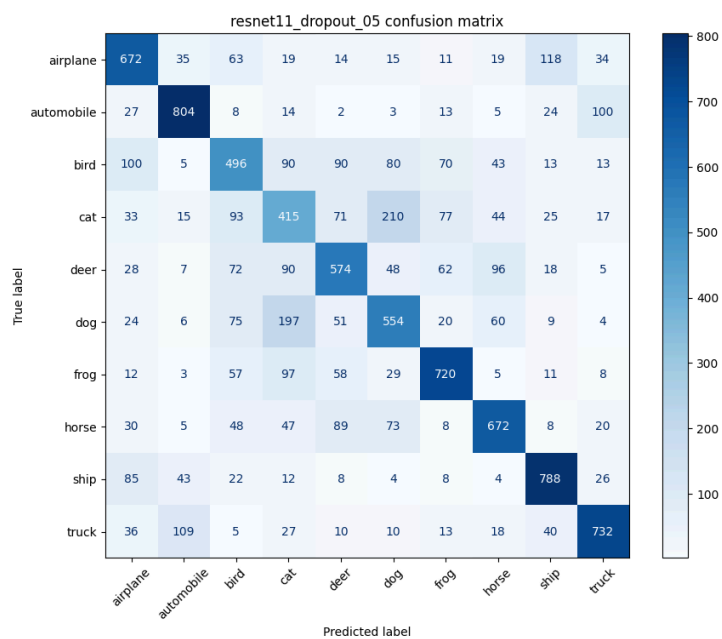
Plots (ResNet 11 - Dropout 0.5)



ResNet 11 - Dropout 0.5 baseline loss curve

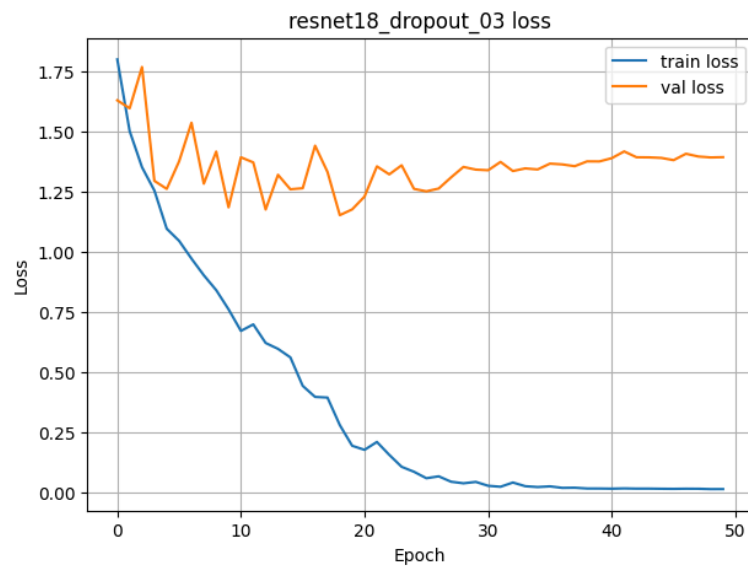


ResNet 11 - Dropout 0.5 baseline accuracy curve

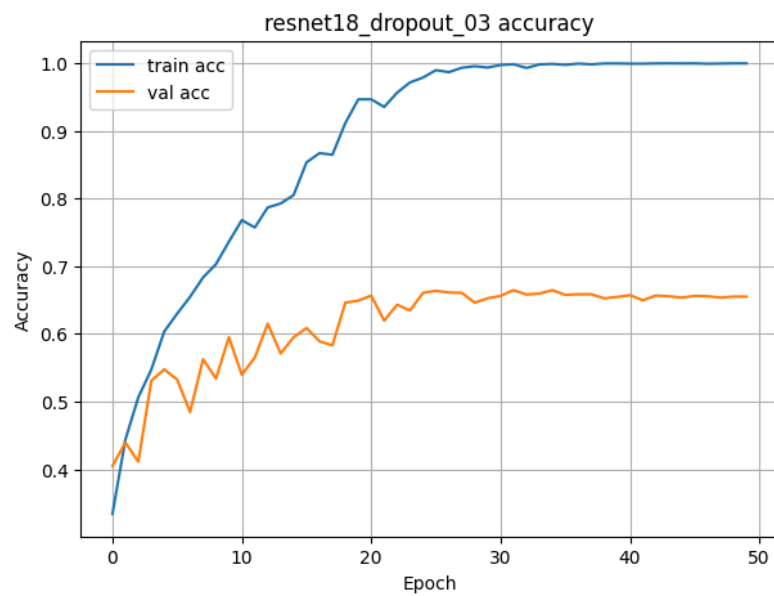


ResNet 11 - Dropout 0.5 confusion matrix

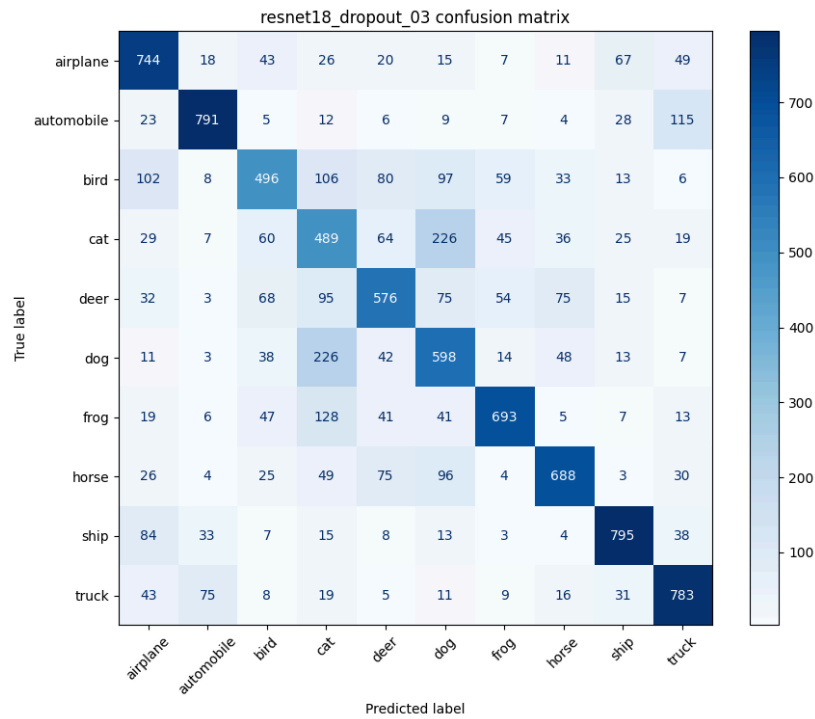
Plots (ResNet 18 - Dropout 0.3)



ResNet 18 - Dropout 0.3 baseline loss curve

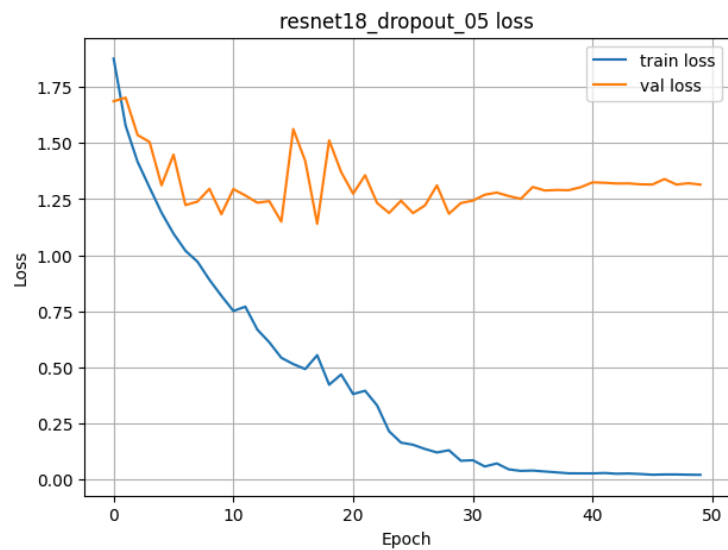


ResNet 18 - Dropout 0.3 baseline accuracy curve

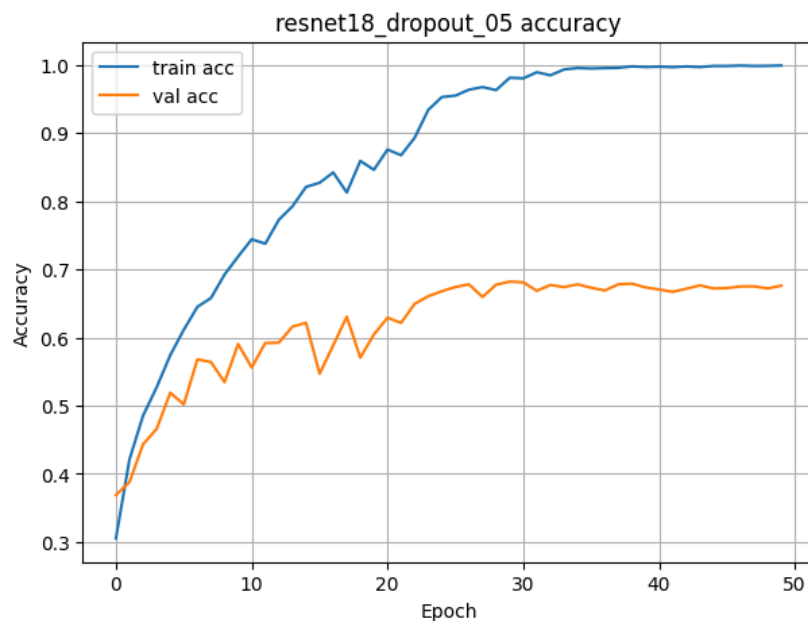


ResNet 18 - Dropout 0.3 confusion matrix

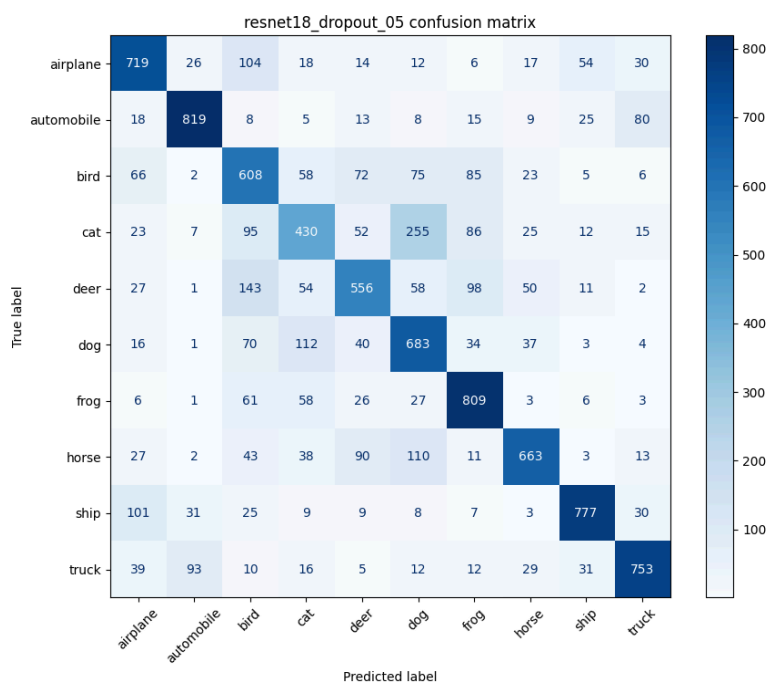
Plots (ResNet 18 - Dropout 0.5)



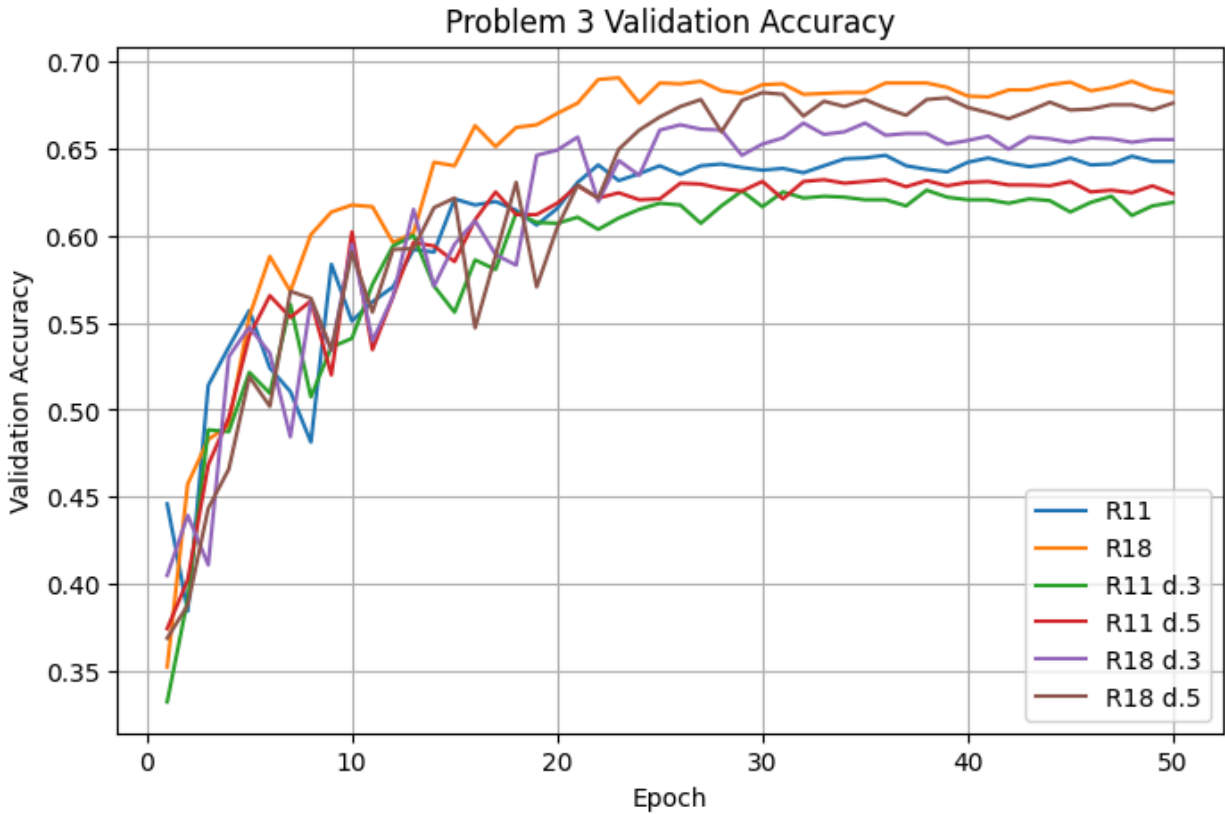
ResNet 18 - Dropout 0.5 baseline loss curve



ResNet 18 - Dropout 0.5 baseline accuracy curve



ResNet 18 - Dropout 0.5 confusion matrix



Model Results

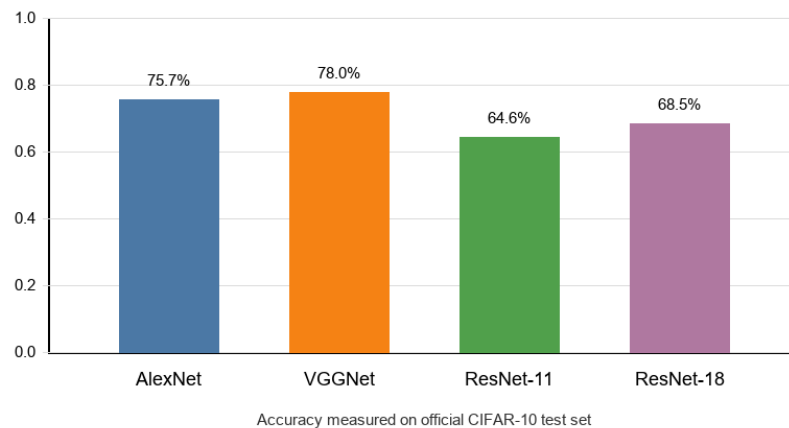
Model	Parameters	Best Val Acc	Test Acc	Best Epoch	Time	Sec/Epoch
resnet11_baseline	604250	64.60%	64.59%	36	62.20 min	74.64 sec
resnet11_dropout_03	604250	62.60%	63.40%	38	33.47 min	40.16 sec
resnet11_dropout_05	604250	63.20%	64.27%	33	45.01 min	54.01 sec
resnet18_baseline	701466	69.05%	68.49%	23	98.01 min	117.62 sec
resnet18_dropout_03	701466	66.45%	66.53%	32	68.58 min	82.30 sec
resnet18_dropout_05	701466	68.20%	68.17%	30	68.62 min	82.34 sec

Final Architecture Comparison

Architecture	Best Variant	Parameters	Test Acc	Best Val Acc	Training Time
AlexNet	dropout_03	612042	75.66%	75.30%	9.51 min
VGGNet	vgg_dropout_05	601258	78.05%	74.90%	41.25 min

ResNet-11	resnet11_baseline	604250	64.59%	64.60%	62.20 min
ResNet-18	resnet18_baseline	701466	68.49%	69.05%	98.01 min

Best Test Accuracy by Architecture



Final comparison of best test accuracy by architecture

Overall, adapted VGGNet performed best on test accuracy. Modified AlexNet was close behind and trained much faster. ResNet-18 improved over ResNet-11, but both compact ResNet models underperformed the AlexNet and VGG models in the fast CPU setup. Full CIFAR-10 training with GPU acceleration would likely improve the ResNet results most.

Full Homework Comparison Plot

The plot below compares the final test accuracy for every saved model variant across Problems 1, 2, and 3.

All Homework 1 Variants: Final Test Accuracy

